

Transformers.

Notes on Transformers taken while going through Jay Alammar's [blog posts](#), Sebastian Ruder's [blog posts](#), and the HuggingFace NLP [course](#).

A Review of the Neural History of Natural Language Processing (upto ~2017)

Blog post from Sebastian Ruder on the history of neural NLP: [link](#).

Word Embeddings

This section summarizes major word-embedding approaches upto the advent of Transformers.

TODO: Sebastian Ruder's blog on word embeddings: [Part 1](#), [Part 2](#), [Part 3](#), [Trends and future directions](#) (cf. 2017).

One-Hot Encoding:

Simplest form of word $\rightarrow$ vector model is one-hot encoding, though it results in sparse vector representations. Word2Vec results in dense vector representations.

TF-IDF:

Before Word2Vec, apart from one-hot encoding, TF-IDF was one of the the most common way to represent words:

$\text{tf-idf}(\text{term}, \text{document}) = \text{tf}(\text{term}, \text{document}) \cdot \text{idf}(\text{term})$

$$\text{tf}(\text{term}, \text{document}) = \frac{n_i}{\sum_{k=1}^V n_k}$$

where, n_i represents the frequency of term i in the document, and V is the vocabulary size. $\text{idf}(\text{term}) = \log \frac{N}{n_{t_i}}$

where, N represents the total number of documents and n_{t_i} represents the number of documents that contain the term i .

Word2Vec

First neural embedding model for learning distributed representations.

Each word is represented with a fixed-size float embedding vector.

Three main ideas:

1. **Continuous Bag of Words (CBOW)**: Predict a *word* given the *context*.

Two types of context:

1. Single-word Context

2. Multi-word Context

2. **Skip-gram**: Predict the *context* given a *word*.
3. **Negative Sampling** Instead of training the classifier on all + word pairs: $(\text{word}_i, \text{word}_{i+1})$ derived in CBOW or in Skipgrams, train the classifier on word pairs: $(\text{word}, \text{random})$ as well.

Speeding up training: Projecting onto the entire vocabulary to select the predicted word is an expensive operation. Instead, train a logistic regression classifier for pairs of words: nearby word pairs are positives while far away word pairs are negatives.

Continuous Bag of Words Single-word Context: Given a word, predict a word: $p(w_i|w_{i-1})$.

Multi-word Context: Given several surrounding words, predict a word: $p(w_i|w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$.

Skip-grams Flip the Multi-word Context CBOW model.

Given a word, predict the surrounding words: $p(w_{i-2}|w_i)$, $p(w_{i-1}|w_i)$, $p(w_{i+1}|w_i)$, $p(w_{i+2}|w_i)$.

References

1. The Illustrated Word2Vec: [link](#). Jay Alammar.
2. On Word Embeddings: [part1](#), [part2](#), [part3](#). Sebastian Ruder.

Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention)

Neural Machine Translation Models

- Seq2Seq models generally consist of an **encoder** and a **decoder**. The encoder processes each item in the input sequence and compiles the information it captures into a vector (called the **context**).
- After processing the entire input sequence, the encoder sends the context over to the decoder, which begins producing the output sequence item by item.
- The encoders and decoders are generally recurrent neural nets - **RNNs** - such as **LSTMs** or **GRUs**.
 - Input: vectorized representation of a **word** (say, a Word2Vec embedding) and a **hidden state** at each timestep.

- Output: vectorized **output** and the **hidden state** for the next timestep.
- The hidden state of the encoder at its last timestep is the context vector, which is passed to the decoder.
- The decoder also maintains a hidden state that it passes from one time step to the next.

Attention

The **context** vector turned out to be a bottleneck for NMTs making it difficult to process long sentences.

Instead of sending only the **context** into the **decoder**, the **Attention** mechanism is able to focus on all relevant parts of the input sequence. Attention allows the **decoder** to look at the context output by the encoder at different timesteps.

Major upgrades from NMTs:

1. The **encoder** passes a lot more data to the **decoder** (passing all **hidden states**, instead of only the last one).
2. At each timestep, the **decoder** parses the **hidden states** of the **encoder** and multiplies each state by a softmax-ed **score** for the timestep, thus amplifying the **hidden states** with high scores, and drowning out **hidden states** with low scores.

Decoder:

1. At each timestep, the **decoder** calculates a **score** for each **hidden state** of the **encoder** and then multiplies these **hidden states** by its **score**.
2. This results in a **context** vector which has amplified relevant information for this particular timestep from all the **hidden states** of the **encoder**.
3. This **context** is concatenated with the **hidden state** of the **decoder** and used as input to a jointly trained MLP that outputs a **word**.

Reference: Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention): [link](#).

A few thoughts from Seb Ruder

- Pretrained word embeddings (word2vec/ GLoVe) only incorporate previous knowledge in the first model layer while the rest of the downstream network is trained from scratch. ELMo/ BERT/ GPT flipped this: use pre-trained embeddings with linear/ shallow networks trained from scratch.

- Just like ImageNet, pre-training on a large corpus of text enables the model to learn general language understanding, which can be fine-tuned on specific tasks with very little data.
- Language modeling is a good pre-training task.
 - Predicting the most probable next word requires an ability to express grammatical syntax and also model semantics.
 - It is an unsupervised and can be trained on massive amounts of data.
 - It has been shown to capture many facets of language relevant for downstream tasks, such as long-term dependencies, hierarchical relations, and sentiment.
 - Doing well on language modeling requires what could be considered *world knowledge* or *common sense*.
 - Opens the door to developing models for previously under-served languages. For very low-resource languages where even unlabeled data is scarce, multi-lingual language models may be trained on multiple related languages at once.
 - LLMs trained on language modeling as a pre-training task have been empirically shown to be extremely sample-efficient, achieving good performance with only hundreds of examples and are even able to perform zero-shot learning.

References

1. The Illustrated BERT, ELMo, and co. [jay-alammar-blog](#).
2. NLP's ImageNet moment has arrived [seb-ruder-blog](#).
3. ChatGPT.

Reading is becoming a distraction. Revisit and go over remaining articles later: [illustrated-gpt2](#), [how-gpt3-works](#), [explaining-transformers](#), [illustrated-retrieval-transformer](#).